Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Project Identification

Date	23 October 2026	**Team ID**	Piyush-Balodiya-GenAI-2026
Project Name	ConstructAI	**Maximum Marks**	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (Web App)
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
ConstructAI/
├── backend/
│   ├── routes/              # API Route Controllers (Auth, Planning, QC)
│   ├── schemas/             # Pydantic Schemas & DTO Validation Models
│   ├── services/            # Groq LPU Vision Services & Business Logic
│   ├── auth.py              # JWT Token Handling & Password Security
│   ├── config.py            # Environment Configuration & App Settings
│   ├── constructai.db       # SQLite Database File (WAL Mode)
│   ├── database.py          # SQLAlchemy Session & Engine Configuration
│   ├── main.py              # FastAPI Application Entrypoint
│   ├── requirements.txt     # Python Dependencies & Frameworks
│   ├── .env                 # Environment Secrets (Local Configuration)
│   └── .env.example         # Template Environment Variables
│
└── frontend/
    ├── src/
    │   ├── api/             # Axios API Client & Endpoints
    │   ├── assets/          # Icons, SVGs & UI Images
    │   ├── components/      # Reusable UI Components
    │   ├── utils/           # Formatters & Helper Utilities
    │   ├── views/           # Dashboard Pages & Modules
    │   ├── App.jsx          # React Router & Core Layout Wrapper
    │   └── main.jsx         # React DOM Mounting Entrypoint
    ├── index.html           # HTML Template Entry
    ├── package.json         # Node.js Dependencies & Run Scripts
    ├── tailwind.config.js   # Tailwind CSS Styling Tokens
    └── vite.config.js       # Vite Bundler & Dev Server Config
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	**https://constructai-virid.vercel.app/** Live Production Deployment on Vercel Global Edge Network
Repository Link	**https://github.com/piyushprajapat6080/ConstructAI** Complete Source Code with all submission folders and history
Hosting Platform / Provider	Vercel (Frontend Client) + Cloud Container Staging (FastAPI Backend)
Login Credentials (Demo)	**Role:** Administrator **Email:** `admin@constructai.internal` **Password:** `AdminAccessKey#2026`
Demo Video Link	**https://drive.google.com/file/d/1eA7C_geC51IBzPd63hh8fu_gWicvW4cP/view?usp=sharing** Video walkthrough demonstrating live crack detection and BOQ calculation

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- **Operating System:** Windows 10 / Windows 11 (64-bit)
- **Python:** Version 3.10 or higher | **Node.js:** Version 18.x or higher & npm
- **Git:** Installed & configured in Windows PowerShell / Command Prompt
- **Groq API Key:** Required for LPU vision inference (configured via .env file)

Step-by-Step Execution (Windows PowerShell / CMD):

Step 1 (Clone Repository):

`git clone https://github.com/piyushprajapat6080/ConstructAI.git` & `cd ConstructAI`

Step 2 (Backend Virtual Environment & Dependencies):

`cd backend`

`python -m venv venv` & `. env\Scripts\Activate.ps1` *(Or activate.bat in CMD)*

`pip install -r requirements.txt`

Step 3 (Environment Configuration):

`Copy-Item .env.example .env` *(Configure GROQ_API_KEY & JWT_SECRET_KEY in .env)*

Step 4 (Start FastAPI Backend Server):

`uvicorn main:app --reload --host 127.0.0.1 --port 8000`

Backend API running at http://127.0.0.1:8000 (API Docs at /docs)

Step 5 (Frontend Installation & Launch):

Open a new PowerShell terminal window:

`cd ConstructAI rontend` & `npm install` & `npm run dev`

Access the web dashboard in browser at http://localhost:5173

Step 5: Known Issues / Limitations

Documented technical constraints, third-party limits, and engineering workarounds implemented in ConstructAI:

S.No	Known Issue / Limitation	Workaround / Status
1	**Cloud Free-Tier Cold Starts:** Render cloud container spins down after inactivity, causing a 40–50s delay on the very first API request.	Automated health-check keep-alive cron job configured to ping backend every 10 minutes, maintaining warm worker pools during evaluator grading sessions.
2	**Windows PowerShell Execution Policy:** Running `. env\Scripts\Activate.ps1` may be restricted by default Windows execution policies.	Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in PowerShell prior to activation, or use Command Prompt (`activate.bat`).
3	**High-Resolution Drone Orthomosaics (>25MB):** Ultra-high-resolution drone site images exceed LLM multimodal input limits.	Client-side canvas downsampling pipeline automatically compresses images to 1080p resolution before transmission to Groq LPU Vision API.
4	**Concurrent Multi-Site Database Writes:** Default SQLite locks during high concurrent write bursts from multiple site audits.	Write-Ahead Logging (WAL mode) is permanently activated in `database.py` with connection pooling; production architecture seamlessly upgrades to PostgreSQL.